



Advanced Credit Card Fraud Detection with Multi-Model on IBM z17

This document provides an example on how to build an advanced fraud detection multi-model on IBM z17 and deploy AI models using a Linux on Z environment, while making use of open-source frameworks, Machine Learning for IBM z/OS (MLz), and more. Within this document, there are various phases of the AI lifecycle included.

Key Words: IBM Synthetic Data Sets, XGBoost Model, PMML, BERT Model, ONNX, Machine Learning for IBM z/OS (MLz)

Abstract

Advances in AI technologies are enabling significant improvements in fraud detection and prevention to meet stringent SLA requirements. However, Banking and Financial organizations are only able to run a fraction of transactions due to throughput and latency constraints with their fraud detection systems.

This solution demonstrates how to train, evaluate, and deploy fraud detection models leveraging Machine Learning and Encoder Models, such as XGBoost and Bert models, along with containerized AI model serving solutions. It uses an Ensemble AI approach with Machine Learning and Encoder Large Language Models for advanced credit card fraud detection. The process starts off by leveraging [IBM's Synthetic Data Set](#) which contains various credit card transactional data, such as User, Time, Amount, Merchant Name, Merchant City, and Zip. The labeled dataset enables the use of supervised learning techniques to train models that can identify fraud patterns.

Once the dataset is prepared, users train multiple machine learning models using Scikit-learn within a Jupyter Lab environment. The training process involves data preprocessing, such as standard scaling. The pipeline then trains and evaluates multiple models, such as XGBoost and BERT models, applying hyperparameter tuning to optimize performance. The models are ranked based on precision, recall, f1-score, and support.

After training, the XGBoost model is exported into a Predictive Model Markup Language (PMML) format and the BERT model is exported into an ONNX format. These models are then deployed on Machine Learning for IBM z/OS (MLz). This demonstrates how multiple AI models can be trained to provide an AI-powered fraud detection solution that can seamlessly embed into business applications, ensuring security and reducing financial risks.

Please Note: All transactions will go through the XGBoost model; however, the Bert-base pipelines are only involved in cases where the pre-set probability score threshold (set by the user) of less than 0.8 is determined by the XGBoost model. This is to further optimize our model prediction scores to predict whether the transaction is fraudulent, allowing the application to take appropriate action, such as approving, flagging, or blocking the transaction.

Architecture

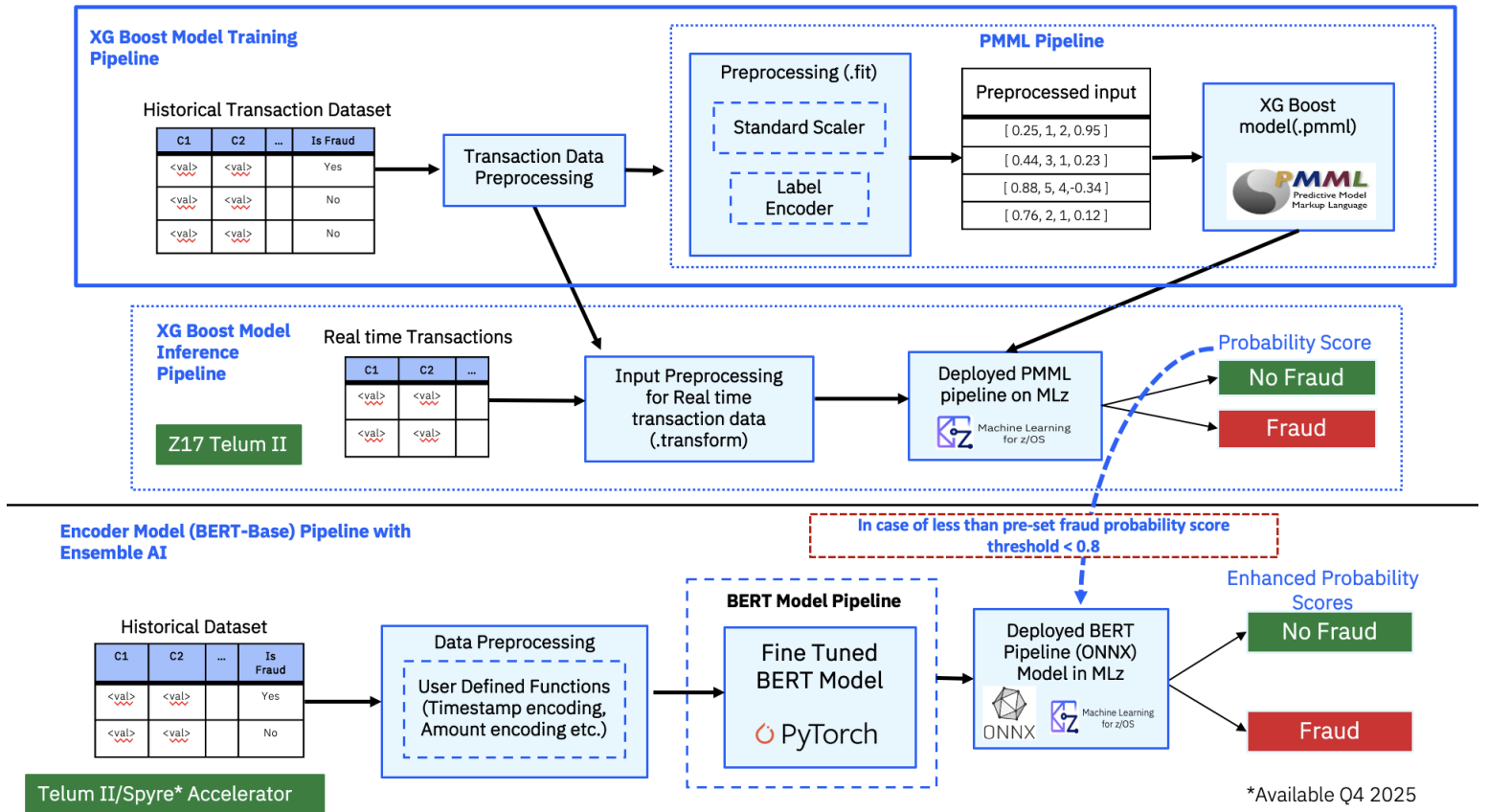
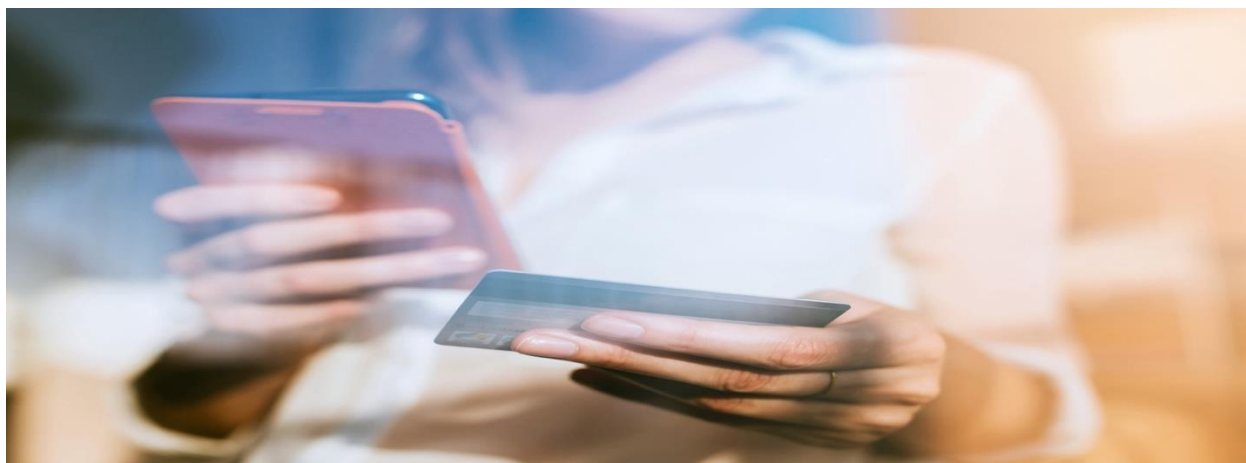


Table Of Contents

AI Model Training.....	5
AI Model Deployment.....	8
AI Model Analysis.....	14



AI Model Training

This section provides a reference overview of how AI models are typically trained, from data preparation and feature engineering to model selection and evaluation. Even if you already have a trained model, understanding the process can be valuable for validation, retraining, or adapting to new use cases.

We'll walk through a scenario to illustrate how one might approach training, which can be helpful for those curious about how it was done. Specifically, we will train an XGBoost and a BERT model and build an inference pipeline using the provided XGBoost_training.ipynb and Bert_training.ipynb Jupyter notebooks. Simply point the notebooks to your dataset and run them to generate your AI models. These trained models can then be deployed seamlessly with MLz.

Technology Used:

- z/OS Container Extension (zCX), Python, Jupyter Notebook, IBM Machine Learning for z/OS

Dataset Guidance:

- The dataset used in this guide is the "Credit Card Fraud" [IBM Synthetic Data Sets](#).

Required Features:

- User (integer) – unique ID for user making transaction
- Card (integer) – unique ID for credit card
- Year (integer) – year of the transaction
- Month (integer) – month of the transaction
- Day (integer) – day of the month of the transaction

- Time (integer) - time of the transaction (HH:MM)
- Amount (float) – dollar amount of the transaction
- Use Chip (string) – the type of transaction
- Merchant Name (integer) – unique ID for merchant name
- Zip (integer) – zip code of the transaction

Access Jupyter Notebook

1. Clone Credit-Card-Fraud-Multi-Model GitHub Repo to your local machine.
2. Run Jupyter to access XGBoost_training.ipynb and Bert_training.ipynb training notebook files.

To get started with the installation of the Jupyter notebooks via the zCX environment, follow the ["How To Get Started"](#) notes.

Provide Data

1. Access your input dataset (.csv file) and place it in your working directory.
2. Add input data to Jupyter notebooks (XGBoost_training.ipynb and Bert_training.ipynb).



```

# Import the pandas library for data manipulation
import pandas as pd
import xgboost as xgb
import numpy as np
import warnings
from sklearn2pmml import PMMLPipeline, sklearn2pmml
warnings.filterwarnings("ignore")

# Load the data from a CSV file into a DataFrame
tdf = pd.read_csv('CCF_data/card_transaction.v1.csv', index_col=False)

```

[3] Python

Model Training

1. Step through and run all cells within Jupyter notebook (XGBoost_training.ipynb and Bert_training.ipynb) within the web browser.

This pipeline automates the entire machine learning workflow, from data ingestion to model selection and evaluation. It begins by loading and validating the dataset, ensuring it contains the necessary features for a supported AI use case, such as fraud detection. The data is then cleaned and preprocessed.

Next, it trains the XGBoost machine learning model, applying feature transformations and hyperparameter tuning using Scikit-learn pipelines. For fraud detection, the models learn patterns from historical transaction data, identifying key fraud indicators such as unusual transaction amounts, location mismatches, and non-chip usage. We will also train on the Bert Encoder model if the fraud probability score is less than the pre-set probability (set by the user) score threshold of 80%. This is where the dataset is then retrained using the Bert language model. The dataset is pre-processed again the cycle for training repeats using a Bert model. Similarly, this cycle is also repeated during the inferencing phase for the XGBoost and Bert models.

Note: This may take several minutes

The trained model is then evaluated based on batch processing through accuracy, precision, recall, and F1-score to determine the effectiveness in distinguishing fraudulent transactions from legitimate ones. By automating this process, the pipeline ensures that organizations can efficiently train, evaluate, and deploy AI models to enhance fraud detection and decision-making.

Access Trained AI Model

1. Once training is complete, navigate and find your XGBoost and Bert AI models for AI model deployment.



AI Model Deployment

After training your model, the next step is deployment using Machine Learning for z/OS (MLz), enabling seamless integration into your enterprise workflows. For users who are beginning directly with deployment or prefer to use pre-trained models, we've included ready-to-use files: `xgboost_fraud_detection.pmml` for the XGBoost model and `bert_model.onnx` for the BERT model. These models can be immediately incorporated into the MLz inference pipeline for production use.

Prerequisites:

- Must have MLz installed

Go to MLz UI

To begin deployment, we'll sign into MLz and start by importing the trained models.

1. Log in with username/password

Machine Learning for IBM z/OS

Log in

Username

spark9

Password

.....

Log in

Need help? Reach out to your system administrator to create an account, recover a username, or reset a password.



Import AI model

The XGBoost model will be imported using the xgboost_fraud_detection.pmml file, while the BERT model will be imported using the bert_model.onnx file. These model files are compatible with MLz and will be used to configure the inference pipeline, enabling efficient scoring and integration into your enterprise applications.

1. Go to "Models" tab
2. Click "Import model"

The screenshot shows the 'Models' tab in the 'Machine Learning for IBM z/OS' interface. At the top, there are navigation tabs: 'Dashboard', 'Models' (selected), 'Deployments', 'Data sources', and 'Runtimes'. Below the tabs is a search bar labeled 'Search Models' and an 'Import model' button. A table lists three models:

Name	Published by	Published on	Type	Latest version	Status	Evaluator
Bert_quantized_ccf_1	spark9	Jun 30, 2025 11:53 AM	ONNX	v1	✓	—
single-relu	spark9	Jun 26, 2025 11:15 PM	ONNX	v1	✓	—
onnx-case-9	spark9	Jun 25, 2025 10:20 PM	ONNX	v2	✓	—

At the bottom of the table, there is a pagination section: 'Models per page: 10' (with a dropdown arrow), '1-3 of 3 items', and '1 of 1 page' (with left and right navigation arrows).

3. Enter model "Name"
4. Choose "Model type"

Note: Choose PMML for the XGBoost model

Note: Choose ONNX for the Bert model

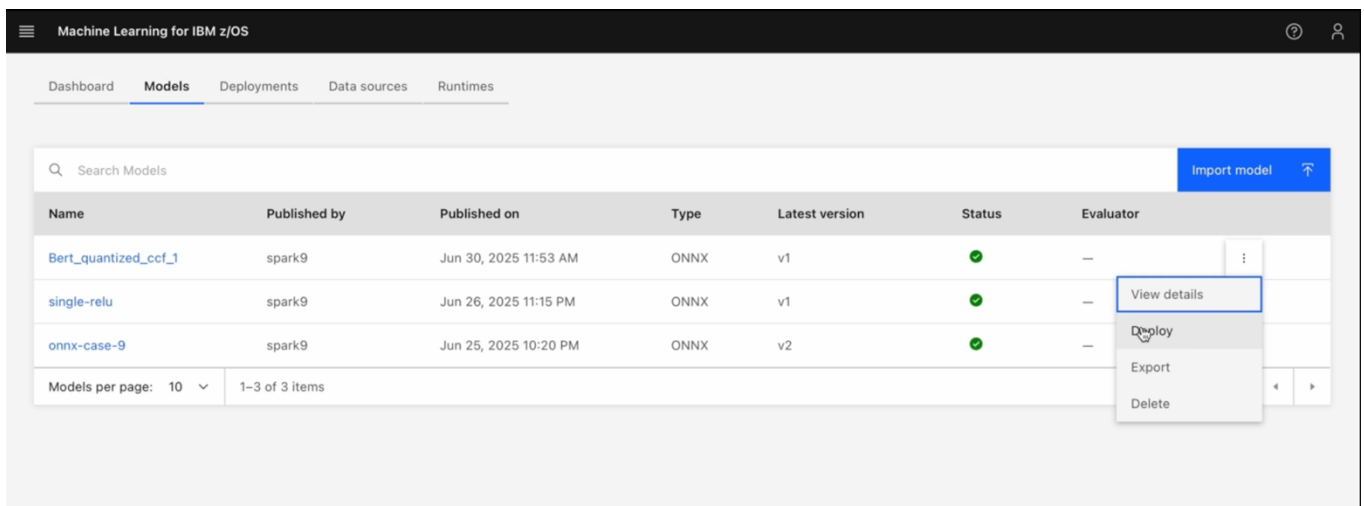
5. Drag and drop model file.
6. Click "Import"
7. Repeat this step for both models – XGBoost model and Bert model

The screenshot shows the 'Import model' page in the 'Machine Learning for IBM z/OS' console. The page has a dark header with the title and navigation icons. Below the header, the breadcrumb 'Models /' is visible. The main content area is titled 'Import model' and contains two tabs: 'From file' (active) and 'From watsonx.ai'. Under the 'From file' tab, there are several input fields: 'Name' with the value 'BERT_CCF', 'Model type' with a dropdown menu set to 'ONNX', and 'Model file' with a dashed box containing the text 'Drop your .onnx file here or browse for a file to upload.' Below this, a file selection box shows 'quantized_model_8bit_1.onnx' with a blue checkmark. At the bottom, there is a 'Custom runtime (optional)' dropdown menu set to 'Select custom runtime'. In the bottom right corner, there are two buttons: 'Cancel' and 'Import'.

Deploy AI Model

With your models successfully imported into MLz, the next step is deployment. MLz provides a structured deployment workflow that guides you through key configuration steps. First, you'll choose the deployment type. Then, you'll select the model version to ensure proper versioning and traceability. Finally, you'll configure the scoring service, which defines how MLz will expose the model for inference. These steps ensure that your XGBoost and BERT models are fully deployed and ready to deliver predictions within your enterprise environment.

1. Go to "Models" tab
2. Click the ellipsis ":" button to view the drop-down menu
3. Click "Deploy"



4. Enter "Deployment name"
5. Choose "Deployment type"

6. Choose "Model version"
7. Choose "Scoring service"

Note: You should choose the correct scoring service based on your application (e.g CICS or REST)

8. Click "Create"

The screenshot shows the 'Create deployment' form in the Machine Learning for IBM z/OS console. The form is titled 'Create deployment' and shows the model 'Bert_quantized_ccf_1'. The deployment name is 'BERT_DEPLOY'. The deployment type is 'Online'. The model version is '1'. The serving ID is 'Fraud_serving_ID'. The scoring service is 'zburns-scoring-service (9.12.41.250:32008)'. There are two checkboxes: 'Use on-chip AI acceleration' and 'Enable micro-batching', both of which are unchecked. At the bottom right, there are 'Cancel' and 'Create' buttons.

Machine Learning for IBM z/OS

Models /

Create deployment

Model: Bert_quantized_ccf_1

Deployment name
BERT_DEPLOY

Deployment type ⓘ
Online

Model version
1

Serving ID (optional) ⓘ
Fraud_serving_ID

Scoring service (standalone or cluster) ⓘ
zburns-scoring-service (9.12.41.250:32008)

☐ Use on-chip AI acceleration ⓘ

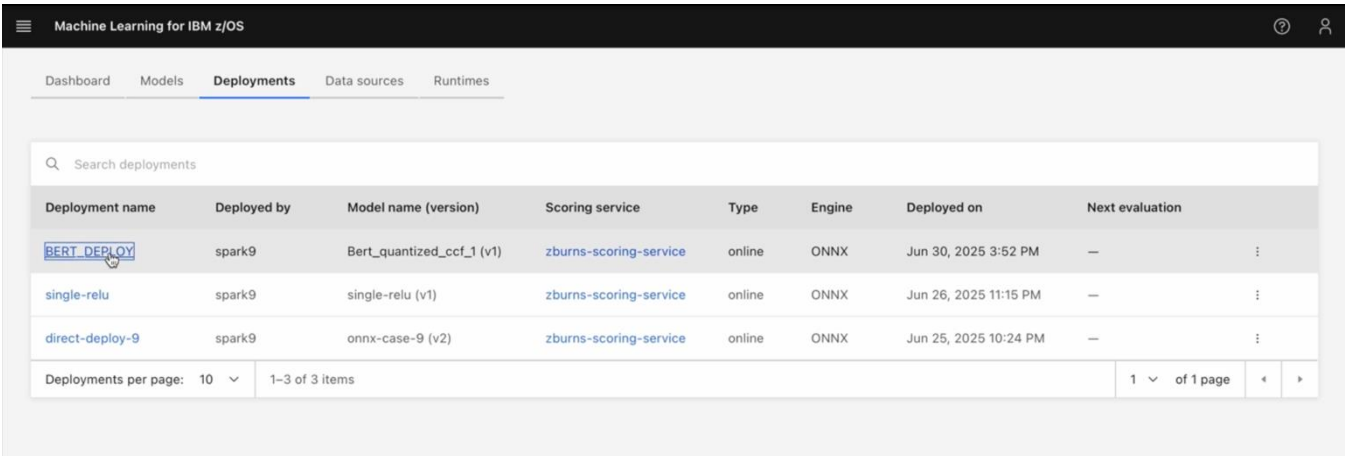
☐ Enable micro-batching

Cancel Create

View Deployed AI Model

After deploying your AI model in MLz, it's important to review the deployment details to ensure everything is configured correctly and running as expected. Checking these details helps validate that the model is correctly set up for inference, supports troubleshooting, and ensures alignment with your operational requirements.

1. Go to "Deployments" tab
2. Click on the "Deployment name" of your deployed model



Machine Learning for IBM z/OS

Dashboard Models **Deployments** Data sources Runtimes

Search deployments

Deployment name	Deployed by	Model name (version)	Scoring service	Type	Engine	Deployed on	Next evaluation
BERT_DEPLOY	spark9	Bert_quantized_ccf_1 (v1)	zburns-scoring-service	online	ONNX	Jun 30, 2025 3:52 PM	—
single-relu	spark9	single-relu (v1)	zburns-scoring-service	online	ONNX	Jun 26, 2025 11:15 PM	—
direct-deploy-9	spark9	onnx-case-9 (v2)	zburns-scoring-service	online	ONNX	Jun 25, 2025 10:24 PM	—

Deployments per page: 10 1–3 of 3 items 1 of 1 page

3. View deployment details

Machine Learning for IBM z/OS

Deployments /

Deployment details

Deployment: BERT_DEPLOY
Model: [Bert_quantized_ccf_1](#) (v1)

Test API call

General

Scoring endpoint
<https://9.12.41.250:32008/impl/v2/scoring/online/c9c19b89-c6d7-42ad-a057-60a5e79f152f>

Serving ID
—

Scoring service
[zburns-scoring-service \(9.12.41.250:32008\)](#)

Scoring timeout in milliseconds
—

API specification
[Download](#) API specification for scoring endpoint.

Number of invocations
0

Deployed on
Jun 30, 2025 3:52 PM

Published by
spark9

Type
online

Engine
ONNX

Active processor
Telum II

Average elapsed time
0 ms

Next evaluation
—

Maximum batch size
—

Maximum latency in milliseconds
—

API documentation
See [API documentation](#) for details

Schemas

Table | JSON | JSON schema | Copybook

Input schemas
View the input data attributes and values for this model.

Output schemas
View the output data attributes and values for this model.

Attribute	Type	Optional?
-----------	------	-----------

Attribute	Type	Optional?
-----------	------	-----------

15



AI Model Analysis

With the AI models deployed, the next step is to analyze their performance and predictions. In this phase, we'll use the XGBoost and BERT inference pipelines to evaluate credit card transaction events for fraud detection. MLz enables you to generate scoring endpoints, which allow you to send data to the models and receive predictions. By running scoring jobs and inspecting the output, you can validate model accuracy and behavior. This analysis is essential for understanding how well the models are performing and identifying opportunities for refinement. With scoring endpoints in place, the final step is model prediction, using these endpoints to make predictions that drive actionable insights in your enterprise applications.

Prerequisites:

- Must have MLz installed
- Must have Python installed
- Must have Git installed

Get Model Details For Inferencing

Once the model is deployed in Machine Learning for z/OS (MLz), you'll need to access its details to begin making predictions. This can be done by navigating to the Deployments tab and selecting the name of your deployed model. From there, you'll find key configuration information, including the scoring endpoint URL. Copying this endpoint is essential, since it serves as the interface through which you send data to the model and receive predictions. Whether you're integrating with an application or testing manually, this endpoint is your gateway to real-time or batch inference.

1. Go to MLz UI
2. Go to "Deployments" tab
3. Click on the "Deployment name" of your deployed model
4. View Deployment details
5. Copy "Scoring endpoint"

The screenshot shows the 'Deployment details' page in the Machine Learning for IBM z/OS interface. The page header includes a hamburger menu, the title 'Machine Learning for IBM z/OS', and user icons. Below the header, the breadcrumb 'Deployments /' is visible. The main title is 'Deployment details', and a 'Test API call' button is in the top right. The deployment information is as follows:

- Deployment: BERT_DEPLOY
- Model: Bert_quantized_ccf_1 (v1)

The 'General' section contains the following details:

- Scoring endpoint:** <https://9.12.41.250:32008/iml/v2/scoring/online/c9c19b89-c6d7-42ad-a057-60a5e79f152f> (Copied! icon)
- Number of invocations:** 0
- Average elapsed time:** 0 ms
- Serving ID:** —
- Deployed on:** Jun 30, 2025 3:52 PM
- Next evaluation:** —
- Scoring service:** [zburns-scoring-service \(9.12.41.250:32008\)](#)
- Published by:** spark9
- Maximum batch size:** —
- Scoring timeout in milliseconds:** —
- Type:** online
- Maximum latency in milliseconds:** —
- API specification:** [Download](#) API specification for scoring endpoint.
- Engine:** ONNX
- API documentation:** See [API documentation](#) for details
- Active processor:** Telum II

The 'Schemas' section has tabs for 'Table', 'JSON', 'JSON schema', and 'Copybook'. Below these are two sections: 'Input schemas' and 'Output schemas', each with a table to view data attributes and values for the model.

Attribute	Type	Optional?

Attribute	Type	Optional?

Analyze Credit Card Events

Creating a scoring endpoint within MLz is a critical step in operationalizing your AI models. Once generated, this endpoint serves as the connection between your deployed model and the inference pipeline, enabling predictions. By integrating the scoring endpoint into your pipeline, you ensure that credit card event data can be automatically sent to the model for fraud detection analysis. This setup not only streamlines the prediction process but also allows for scalable, consistent, and secure inference across enterprise applications.

1. Generate integrated scoring endpoint in XGBoost and Bert inference pipelines. This code will be found in the ensemble_inference_MLz.ipynb jupyter notebook.

Ensemble pipeline activation

Confidence Score and Model Pipeline

A **confidence score** is quickly calculated to assess the confidence level of the prediction from the **XGBoost model**.

- If the confidence score is **greater than 0.8**, it indicates a strong prediction, and no further action is required.
- If a **lower confidence** is observed (below 0.8), the input data is passed through a **BERT model pipeline** to obtain more accurate results with a higher confidence.

This approach ensures that **XGBoost** handles high-confidence cases efficiently, while **BERT** is used for more uncertain predictions, yielding stronger results.

```
[25] def pred(val):  
    ''' Function for checking the fraud/no-fraud case '''  
    if val == 1:  
        display(HTML('<div style="color: #d9534f; font-size: 16px; font-weight: bold; background-color: #f8d7da; padding: 10px; border-radius: 5px;">  
    else:  
        display(HTML('<div style="color: #28a745; font-size: 16px; font-weight: bold; background-color: #d4edda; padding: 10px; border-radius: 5px;">
```

Python

```
[26] predicted_prob=list(predictions.values())
```

Python

```
[28] # checking the confidence of XGBoost model  
if max(predicted_prob)>0.8:  
    print("Met the confidence level, hence bert pipeline is not activated.")  
    pred(predicted_prob.index(max(predicted_prob)))  
  
#Bert model pipeline activation  
else:
```

The inference pipeline shows the latest predictions that the AI models have made. This is done when one payload to the endpoint is being passed for prediction from the XGBoost model. The second endpoint is invoked when the pre-set probability score threshold (set by the user) of less than 0.8 is determined providing the final credit card results from the Bert model.

Make Predication

Now that the model is deployed and the scoring endpoint is ready, we can start making predictions. We'll take input data, like credit card transactions, and send it to the model using a test API call. This call goes through the scoring endpoint, which is like a gateway to the model.

The model then looks at the data and gives back a prediction score, helping us understand if a transaction might be fraudulent.

1. Given Input Data will be in following format:

Given Input Data

```
{
  "User": 359,
  "Card": 1,
  "Year": 2000,
  "Month": 10,
  "Day": 2,
  "Time": "08:49",
  "Amount": "$394.00",
  "Use Chip": "Swipe Transaction",
  "Merchant Name": "-6161792371494728879",
  "Merchant City": "Mexico City",
  "Merchant State": "Mexico",
  "Zip": "",
  "MCC": 3389,
  "Errors?": ""
}
```

2. Check prediction scores.

Prediction Results

Probability (Non-Fraud) : 0.06719186902046204
Probability (Fraud) : 0.9328081011772156

Alert!! The transaction is fraud